

# From PDF to Knowledge graph: A Fine-tuned LLM vs RAG-KG vs Zero-shot LLM comparison

Natural Language Processing and Text Mining - Project 24

Code available at: [https://github.com/raphsa/pdf\\_to\\_KnowledgeGraph\\_LLM](https://github.com/raphsa/pdf_to_KnowledgeGraph_LLM)

Raffaele Sali  
rsali25@student.oulu.fi

Martina Vianello  
mvianell25@student.oulu.fi

Beatrice Zani  
bzani25@student.oulu.fi

**Abstract**—This project presents a modular pipeline that transforms complex, domain-specific documents into structured, queryable knowledge for grounded language model applications. The workflow begins with PDF ingestion and parsing, extracting text and tables with section-level metadata. A domain ontology defines key entities and relations in food, nutrition, and health, guiding entity recognition and relation extraction to generate validated triples. These are stored in a knowledge graph, enabling structured exploration and fact retrieval. From this graph, a grounded instruction–response dataset is automatically generated, supporting fine-tuning and evaluation of large language models. The system includes Zero-shot, RAG-KG, and fine-tuned baselines, evaluated for factuality, faithfulness, and hallucination control. Finally, a Gradio demo illustrates real-time question answering with source-grounded explanations. The framework demonstrates a scalable approach to integrating structured semantic knowledge with interpretable language model reasoning.

**Index Terms**—Knowledge Graph, NLP, LLM, RAG-KG, Text Mining, Fine-Tuning

## I. INTRODUCTION

Knowledge representation in structured form has become an increasingly important pillar in the development of intelligent systems. A key paradigm is the Knowledge Graph (KG), which represents real-world entities as nodes and the relationships between them as edges, enabling both human-readable and machine-processable semantics. Parallel to this, the advent of large language models (LLMs) has transformed how unstructured textual data is used and processed. However, while LLMs excel at language understanding, they are prone to producing content that is not grounded in factual evidence or reliable structure, commonly termed *hallucinations*. Integrating knowledge graphs with LLMs can help ground their outputs in structured knowledge and reduce unanchored generation. Documents in PDF format remain one of the most common mediums in many domains (academic, technical, regulatory). Yet, extracting structured knowledge from PDFs poses substantial technical challenges: the text may be unstructured, include tables and figures, hierarchical headings, and embedded images.

In this context, our project addresses the following research problem: how to extract structured knowledge from a

domain-specific PDF (in the food domain), construct a knowledge graph (KG), and use that graph to generate grounded instruction-response pairs in order to fine-tune an LLM, and then compare this fine-tuned model with two baselines: (i) zero-shot LLM, and (ii) retrieval-augmented generation over the KG (RAG-KG).

Recent dietary guidelines such as the Finnish “National Nutrition Recommendations 2024” emphasise not only individual health, but also the planetary health consequences of food production and consumption. For instance, food consumption in Finland has been estimated to account for around 20% of the annual climate impacts of the average consumer, and diets aligned with nutritional and sustainability goals can potentially reduce the climate footprint by over one-third. These guidelines are produced through large-scale surveys, modelling of consumption patterns, and systematic reviews of nutritional and environmental research. As the main source of domain knowledge, this project relies on the “Sustainable Health from Food – National Nutrition Recommendations 2024” report by the Finnish Institute for Health and Welfare [1]. This document provides an evidence-based synthesis of food and health recommendations, yet it exists primarily as a human-readable PDF, where valuable knowledge is distributed across sections, tables, and figures. In this project, we take this complex PDF as our source document and convert its contents into structured knowledge within the food and nutrition domain. By doing so, we establish a pipeline that enables downstream tasks—such as fact-based instruction–response generation and fine-tuning of large language models—and allow for systematic comparison between grounded and ungrounded model settings.

## II. METHODOLOGY

The methodology of this project follows a modular pipeline designed to extract structured knowledge from a complex, domain-specific document and to use it for large language model (LLM) fine-tuning and evaluation. The methodology is organised into several components, each corresponding to one stage of the pipeline:

- PDF ingestion and parsing;
- Domain ontology and ontology draft;

- NER and keyphrase extraction;
- Relation extraction and triple building;
- Knowledge graph construction and storage;
- Instruction-response dataset creation (grounded);
- Fine-tuning setup;
- Evaluation (Quality, Factuality, Faithfulness);
- Hallucination detection and mitigation;
- Delivery and demo.

This section describes each component in detail.

#### A. PDF ingestion and parsing

The PDF ingestion and parsing system was designed to convert a complex, structured PDF document into a machine-readable format suitable for data analysis and information retrieval. The process integrates several Python libraries to detect, extract, and organise both textual and tabular information while preserving the logical structure of the original document. The first stage focuses on identifying and parsing the table of contents (TOC), in order to then get a better division for text chunks. Using PyMuPDF, the entire PDF and the section titled “Contents” or “Table of Contents” is scanned. This section is then cleaned of irrelevant page artefacts such as publisher names or header text. After finding the section, through the use of regular expressions, each line of the TOC is decomposed into a section title and a corresponding page number. The resulting output is normalized and assembled into a tabular form using `pandas`, which produces a structured index of section titles and their starting pages. This index forms the backbone of the document’s hierarchical reconstruction.

In the second stage, the program iteratively traverses the full text of the PDF, page by page, to associate each segment of text with its correct section heading. Each page is read as a series of lines, and pattern matching is used to identify when a new section begins, based on the TOC entries detected earlier. Text that falls between these headings is aggregated and labelled as belonging to that specific section. The final result is a set of text chunks in which every passage is linked to a document section and page number. These are stored in a JSON Lines (`.jsonl`) file to facilitate subsequent analysis and/or indexing.

```
{
  "page": 33,
  "section": "Carbohydrates and fibre",
  "text": "The recommended intake of carbohydrates is 45 to 60% of the total daily energy intake (E%). The recommended intake..."
}
{
  "page": 33,
  "section": "Fat",
  "text": "The recommendations for fat intake emphasise quality, that is, the fatty acid composition of the diet. The recommended fat intake is 25 to 40% of the total daily energy intake (E%). An inadequate fat intake..."
}
```

Fig. 1. Example of parsed text output in JSONL format.

Parallel to text extraction, a third process handles table recognition. The script employs `Camelot` in two configura-

tions, `STREAM` and `LATTICE`, to capture both text-aligned and line-bordered tables. Every detected table is cleaned, converted into a `pandas DataFrame`, and exported as a CSV file, together with metadata indicating the page number, table dimensions, and file path. In this way, textual and tabular data are separated but remain referentially linked to the same document structure. Together, these steps produce a structured dataset from the original PDF: a machine-readable representation of both the narrative and the quantitative content of the report. This enables further computational analysis, such as keyword extraction or thematic clustering.

#### B. Domain schema and ontology draft

To enable structured representation of entities and relations extracted from the source document, a domain-specific ontology was designed for the food, nutrition, and health domain. The ontology defines the conceptual schema for the knowledge graph and provides standardized classes, attributes, and relations linking ingredients, nutrients, dietary guidelines, techniques, health outcomes and environmental impacts.

The ontology was implemented in `YAML` format and assigned a base URI (`http://example.org/food#`) for semantic consistency. It serves as a lightweight conceptual model aligning with `RDF` principles, enabling conversion to `Turtle (TTL)` or `RDF/XML` for graph storage and querying.

Six main classes were defined:

- **ingredient** – physical food items or raw materials (e.g., wheat, fish, legumes) with attributes for group, origin, and environmental impact.
- **nutrient** – chemical or biological components contributing to physiological functions (e.g., protein, vitamin C, omega-3).
- **technique** – preparation or processing methods influencing food properties, safety, or nutrient content (e.g., boiling, fermentation).
- **dietaryGuideline** – official recommendations specifying consumption or preparation guidance for health and sustainability goals.
- **healthOutcome** – measurable health effects or conditions linked to nutrients or dietary behaviors (e.g., cardiovascular risk, bone health).
- **environmentImpact** – quantified environmental effects associated with foods or production processes (e.g., greenhouse gas emissions, water use).

The ontology also defines key relations connecting these entities, such as:

- **hasNutrient**: ingredient → nutrient.
- **usesTechnique**: ingredient → technique.
- **hasGuideline**: ingredient → dietaryGuideline
- **aimsToImprove**: dietaryGuideline → healthOutcome
- **hasEnvironmentalImpact**: ingredient → environmentImpact
- **affectsRiskOf**: nutrient → healthOutcome

Each relation includes domain, range, and cardinality constraints to ensure schema consistency during triple generation.

The ontology acts as a unifying layer between raw textual information and the structured knowledge graph. It ensures that all extracted entities and relations adhere to consistent categories and relation types. The design intentionally balances minimalism (for computational efficiency) and expressivity (for coverage of nutritional, environmental, and health dimensions).

### C. NER and keyphrase extraction

The named entity recognition (NER) and keyphrase extraction stage focused on identifying and normalizing domain-specific entities mentioned in the text. This process aimed at generating a structured list of unique entities, aligned with the previously defined ontology, to create the base of the knowledge graph population.

Firstly, a seed vocabulary was created to list an initial set of example entities extracted from the parsed text. Each text chunk was analyzed and cleaned (for instance, by removing punctuation), then occurrences of ontology classes were detected by using predefined keyword lists and regular expressions. Only the terms appearing in at least two distinct chunks were kept. The resulting list of seeds was then cleaned and refined by removing irrelevant terms and adding missing but semantically important ones. An example of terms saved for the class **ingredient** includes: “alcohol”, “barley”, “beans”, “berries”, “butter”, “cereals”, “cheese”.

The second step consisted in generating the final entities file, where each line contains information about a unique entity found in the extracted text. This process combined the raw text, the seeds created earlier and the ontology to perform entity recognition and normalization. Using *spaCy*, the algorithm scanned the text chunks looking for mentions of the seed examples, then grouped synonyms and merged aliases. For each merged group, the most generic label was selected as the canonical entity name and a unique identifier was assigned. The most important fields stored for each unique mention are:

- **id** - unique identifier created from the ontology class prefix and an incremental index.
- **type** - ontology class associated with the entity.
- **label** - canonical name of the entity.
- **mentions** - list of mentions of the entity found in the text, each including page reference and context.

After this NER pipeline, the resulting file, saved in JSON Lines (*.jsonl*) format, provided the starting point for the next steps of execution such as relation extraction and triple building.

```
{
  "id": "http://example.org/food#ingredient_salt",
  "type": "ingredient",
  "label": "salt",
  "mentions": [
    {
      "page": 5,
      "section": "Preface",
      "text_span": [556, 560],
      "context": "... challenges that remain are excessive salt and saturated fat intakes and insufficient fibre intake."
    },
    {
      "page": 5,
      "section": "Preface",
      "text_span": [931, 935],
      "context": "... vegetable oils, fish and nuts, and limit the use of salt."
    }
  ]
}
```

Fig. 2. Example of entity in JSONL format.

### D. Relation extraction and triple building

Following entity extraction, a rule-based system was implemented to construct a semantic representation of the document in the form of an RDF knowledge graph. This stage transforms the previously identified entities, such as ingredients, nutrients, techniques, or health outcomes, into formally defined relationships, enabling structured querying and reasoning. The process begins by reading the JSONL file containing the detected entities and all their textual mentions. Each record includes the entity’s label, its ontology type (e.g., ingredient, nutrient, healthOutcome), and metadata describing where it appears in the document, such as the page number, section title, and contextual snippet. Every mention is normalized to lowercase and stored alongside its surrounding context to preserve interpretability. To avoid generating relationships that cross semantic boundaries, the mentions are grouped by page and section, and whenever a potential relation between two entities is detected, their co-occurrence within the same sentence is verified. This ensures that relations are inferred only when entities appear in close textual proximity, preserving contextual relevance. In other words, only entities that occur within both the same section and the same sentence are compared, preventing the creation of links across unrelated topics. For example, the section on “Protein” will generate relations exclusively among entities that co-occur in the same sentences within that section. A set of domain-informed relation rules is then applied to each group. These rules specify meaningful pairings between ontology types, for example:

- ingredient → nutrient → hasNutrient
- ingredient → technique → usesTechnique
- nutrient → healthOutcome → affectsRiskOf
- dietaryGuideline → environmentImpact → guidelineTargetsImpact

Each rule represents a plausible semantic relation grounded in the food and nutrition domain. When the program detects a valid type pairing within the same section, it records the relationship as a triple in the format (subject, predicate, object). All generated triples are stored in a Resource Description Framework (RDF) graph using the *RDFLib* library. Each entity is represented as a URI within the namespace, ensuring global uniqueness. In addition, each triple is annotated with an *rdfs:comment* that records its origin (page number and section name), maintaining traceability to the source document. The final graph is serialized in the Turtle (*.ttl*) format.

```
{ex:oils ex:hasNutrient ex:fat ;
  rdfs:comment "Found on page 72, section 'Diet composition'" .

ex:grains ex:hasNutrient ex:protein ;
  rdfs:comment "Found on page 72, section 'Diet composition'" .}
```

Fig. 3. Example of RDF triples in Turtle syntax.

### E. Knowledge graph construction and storage

After generating the domain-specific RDF triples, an ontology integration and querying stage was implemented to enable structured exploration of the extracted knowledge. To manage and query the extracted knowledge graph, Apache Jena Fuseki was employed as a SPARQL endpoint. The server was deployed locally using Docker, ensuring a reproducible and isolated environment for data management. The RDF triples generated from the extraction process were uploaded to a dedicated Fuseki dataset ([http://localhost:3030/nutrition\\_pdf](http://localhost:3030/nutrition_pdf)), allowing direct querying of the structured information.

The interaction with the dataset was performed through the SPARQLWrapper Python library, which allowed programmatic execution of SPARQL queries and seamless integration with data analysis tools. After setting up the database, a series of queries were executed for a sanity check and the results were displayed. For example, some queries retrieved relationships between ingredients and nutrients, associations between food items and environmental impacts, and links between nutrients and health outcomes. A final query extracted entities annotated with the corresponding page numbers from the source PDF, allowing the provenance of each extracted element to be traced back to its original context.

```
{PREFIX ex: <http://example.org/food#>
SELECT ?nutrient ?outcome
WHERE { ?nutrient ex:affectsRiskOf ?outcome .
}
LIMIT 20}
```

Fig. 4. Example of SPARQL query to retrieve 20 nutrient–health relations.

### F. Fact set and contradiction checks

This part of the workflow serves as a bridge between the structured knowledge representation of the RDF graph and the natural-language domain needed for further processing, such as text generation, retrieval, or summarization. The main goal was not simply to extract triples, but to give semantic data a human-readable and context-aware form. The RDF graph, parsed from the ttl file, contains a rich network of factual statements about foods, nutrients, and dietary guidelines, but these statements are stored as machine-level relations that are difficult to interpret without additional processing. By executing a SPARQL query, the program systematically identified all relevant triples that use the project-specific namespace and, where available, attached page-level metadata from the original document. This ensures that every extracted fact can be traced back to its exact source within the report, maintaining transparency and verifiability.

Once extracted, the data were organized into a structured DataFrame to facilitate manipulation and inspection. The subsequent step involved the generation of *paraphrases* for each triple. Instead of presenting knowledge as abstract entities and predicates, the system reformulated them into coherent

English sentences, for example, converting a triple such as “fruit hasNutrient vitamin c” into “fruit contains vitamin c” or “vitamin c is a nutrient found in fruit.” In doing so, the system transforms the RDF graph into a form that can be directly integrated into LLM training, KG retrieval or summaries.

Finally, each enriched record, including the original triple, any available metadata, and its paraphrased sentences, was saved as a JSONL file.

### G. Instruction-response dataset creation (grounded)

To construct a grounded dataset suitable for model fine-tuning, an automated instruction–response generation pipeline that converts atomic facts from the KG into natural language question–answer pairs was implemented. Each fact in the graph, represented as a structured triple (subject, predicate, object), was transformed into multiple semantically diverse question templates, allowing the dataset to capture both factual retrieval and reasoning-based tasks. The generation process was implemented in Python, leveraging rule-based templates parameterized by ontology relations defined earlier. For each triple, the system produced one or more **factoid questions** (e.g., “What nutrient is found in oats?”), **list or comparison queries** (e.g., “List foods rich in vitamin C”), **reasoning prompts** (e.g., “If a recipe uses frying, what environmental impact may increase?”), and **constraint-based instructions** (e.g., “Find vegan foods rich in calcium”).

All generated pairs were grounded in the original source document through explicit page-level citations extracted from metadata, ensuring that the training data remained fully faithful to the reference report. The output format followed a JSONL schema containing the fields **instruction**, **input**, **output**, and **grounding**, where the grounding array records the supporting facts and provenance (subject, predicate, object, page). After generation, the dataset was shuffled and split into training (80%), validation (10%), and test (10%) subsets to prevent information leakage across model evaluation phases. This approach allows for scalable, reproducible creation of high-quality, domain-grounded instruction datasets suitable for both supervised fine-tuning and retrieval-augmented evaluation.

```
{{"instruction": "What nutrient is found in\nnuts?",\n  "input": "",\n  "output": "nuts contains selenium. (Source:\np.49)",\n  "grounding": [{"subject": "nuts", "predicate": "hasNutrient", "object": "selenium",\n    "page": "49", "paraphrases": ["nuts\ncontains selenium.", "selenium is a\nnutrient found in nuts."]}]}}
```

Fig. 5. Example of instruction from the test set.

### H. Baselines: RAG over KG vs. Zero-Shot LLM

The baseline evaluation stage aimed at comparing two different strategies for question answering over the extracted

knowledge: a **retrieval-augmented generation (RAG)** approach grounded in the knowledge graph and a **zero-shot large language model (LLM)** approach without context retrieval. The objective was to compare the effectiveness, efficiency, and contextual grounding of LLM outputs when structured evidence was provided versus when only internal model knowledge was used. Subsequently, the two baseline models were compared with the results obtained from the fine-tuned LLM, in order to assess how the latter behaved with respect to the two reference models.

The choice of the LLM used to construct the two baseline models was guided by a trade-off among cost, performance, and sustained capacity in terms of tokens and request limits, both per minute and per day. This evaluation led to the selection of the Groq API, specifically the **llama-3.1-8b-instant** model.

Both baseline models were prompted using predefined instructions:

- **Zero-shot baseline:** the prompt consisted only of the instruction “*Answer this question: question*”. In this setting, the model was required to generate an answer solely on the basis of its internal knowledge, given a specific question.
- **RAG-KG baseline:** the prompt was composed of two parts: the contextual information retrieved from the knowledge graph, followed by the instruction to answer the question based on the provided context, explicitly citing the page from which the information was extracted.

Context extraction from the knowledge graph, which underpinned the RAG-KG pipeline, was performed by identifying relevant terms using the `en_core_web_sm` model from the `spaCy` library, which allowed to perform basic keyword extraction from user-supplied questions. Once the terms were extracted, bigrams were generated by combining consecutive terms, in order to correctly handle compound expressions (e.g., different types of vitamins). Single terms were first used to identify potential predicates referenced in the question. Subsequently, bigram matches were searched in the knowledge graph using either exact string matching or a maximum edit distance of one, to account for typographical errors. Prioritizing bigram matching allowed single terms belonging to compound entities to be discarded, thereby avoiding false positive matches. The same matching strategy (exact or edit distance-based) was then applied to individual terms, leading to the identification of triples relevant to the question context. The matched facts were then formatted into a structured context for the RAG-KG prompt, represented as a list of bullet points containing subject - predicate - object triples. It was decided not to include text chunks in the retrieved context, as preliminary observations indicated that doing so led the LLM to produce less specific answers and significantly increased the size of the prompt, thereby slowing down the overall pipeline.

Finally, both baseline models were applied to the set of questions in the test split of the instruction-response dataset, generating the predicted answers for evaluation, saved in a JSONL file.

```
{
  "instruction": "What is the main nutrient in salt?",
  "prediction": "Based on the provided information, the main nutrient in salt appears to be sodium. \n\nThis conclusion can be drawn from the fact that sodium is consistently listed as a component of salt across various page numbers (Page 5, 19, 21, 35, 38, 42, 48, 53, 56, 64, 66, 70, 71, 82, 83, 90, 91). It is listed 17 times out of 17, indicating a very strong association between sodium and salt.\n\nPage numbers cited: (17 instances) 5, 19, 21, 35, 38, 42, 48, 53, 56, 64, 66, 70, 71, 82, 83, 90, 91"}

```

Fig. 6. Example of predicted answer of RAG-KG on test set.

```
{
  "instruction": "What is the main nutrient in salt?",
  "prediction": "The main nutrient in salt is sodium. However, I'd like to provide more information. While salt contains other minerals, the primary component is sodium chloride (NaCl), and the main nutrient in terms of percentage by weight is indeed sodium."}

```

Fig. 7. Example of predicted answer of Zero-shot on test set.

### 1. Fine-tuning setup

For the fine-tuning phase, the **Mistral-7B** model was employed, a compact yet high-performing open-weight large language model designed for efficient instruction following and factual reasoning. Mistral was selected due to its balance between computational efficiency and linguistic capability, enabling fine-tuning on consumer-grade hardware while retaining generation quality. To reduce memory overhead and training time, **Low-Rank Adaptation (LoRA)** was applied, which introduced a small number of trainable parameters into existing model layers instead of updating all weights. This approach allowed for fast domain adaptation without compromising the integrity of the pretrained representations. The training data consisted of the instruction–response pairs generated from the knowledge graph, ensuring that all responses were grounded in factual content extracted from the report “Sustainable Health from Food” PDF. Each example included the instruction, its grounded context, and reference sources to encourage citation-aware generation.

Figure 8 illustrates the structure of the prompt used to fine-tune the model. Several system prompts were tested during the different training sessions, with the best results obtained using the prompt shown in the figure. For each row of the test set, the system prompt was combined with the instruction and the expected response.

Training was performed using parameter-efficient fine-tuning (PEFT) methods implemented via the Hugging Face transformers and peft libraries. Hyperparam-

```
[{"role": "system", "content": "You are a helpful assistant that answers questions about food and health relationships based on scientific evidence. Always cite your sources with page numbers."}, {"role": "user", "content": user_message}, {"role": "assistant", "content": output}]
```

Fig. 8. Prompt structure for fine tuning stage

eters were optimized through iterative experimentation to balance training stability, model generalization, and computational efficiency. Several configurations were tested, varying the learning rate, LoRA rank (4, 8, and 16), LoRA alpha (16, 32) and the number of epochs (ranging from 3 to 20) [2]. These experiments allowed us to identify the combination that produced consistent convergence without overfitting or degradation in factual precision. `bf16` precision was used to minimize memory usage and improve numerical stability, AdamW optimizer with a linear warm-up schedule (10% of total steps) was employed to ensure smooth gradient updates during the early training stages. The fine-tuning process ran for twenty epochs, with early stopping based on validation loss to prevent overfitting and unnecessary computation. Careful tuning of these parameters was crucial: the learning rate determined the extent of weight adaptation from the base Mistral model, while the LoRA rank controlled the representational capacity of the adapter layers—higher ranks increasing expressiveness, but also memory cost. Similarly, the epoch count influenced how well the model internalized domain-specific relations without memorizing examples. This parameter sensitivity analysis ensured that the final configuration achieved optimal alignment with the nutrition and sustainability domain while maintaining efficient resource utilization. To further enhance the linguistic robustness and the generalization of the model, paraphrased versions of the training examples were included in the fine-tuning step. This approach exposed the model to semantically equivalent questions expressed in varied syntactic forms, allowing it to generalize beyond fixed phrasings. From a practical perspective, this step is critical in mitigating surface-form dependency, ensuring that the model learns to reason over factual content rather than memorize specific lexical patterns. As a result, the model demonstrated improved tolerance to linguistic variation, answering correctly even when user inputs deviated from the original dataset formulations.

### J. Evaluation

During the evaluation stage, several metrics were employed to assess the performance of the fine-tuned model compared with the two baseline models. Both automatic metrics, whose assessment derives from a computed score, and rubric-based metrics, where evaluation was delegated to an LLM (specifically, `meta-llama/llama-4-scout-17b-16e-instruct`), were developed. The following metrics were used:

- **Automatic Metrics**

- **F1 Score:** computes the harmonic mean between precision and recall at the token level, capturing partial overlap between the predicted and reference answers. The F1 score was used only for factoid instructions.
- **ROUGE (Recall-Oriented Understudy for Gisting Evaluation):** evaluates the similarity between predicted and reference answers based on overlapping n-grams. ROUGE1 (measures overlaps of unigrams), ROUGE2 (measures overlap of bigrams) and ROUGE L scores (measures the longest common Subsequence between the predicted and reference answers) were computed.
- **BLEU (Bilingual Evaluation Understudy):** measures the precision-oriented overlap of n-grams between the generated output and the reference answer. BLEU1, BLEU2, BLEU3 and BLEU4 scores, which differ in the length of the n-grams they consider, were computed.
- **Constraint Satisfaction Rate:** quantifies the proportion of model responses that satisfy predefined structural or semantic constraints, such as the presence of required entities, citations, or formatting rules. This metric was applied only to constraint-based queries, where such conditions are explicitly encoded in the instruction.

- **Rubric-Based Metrics**

- **Relevance:** assesses how directly and appropriately the answer addresses the question.
- **Completeness:** evaluates whether the response includes all key information needed for a correct and exhaustive answer.
- **Citation Correctness:** measures whether the answer appropriately cites the source from which the information was derived, and whether the cited reference is accurate.

The rubric-based metrics were implemented by providing the evaluation LLM (`meta-llama/llama-4-scout-17b-16e-instruct`) with a dedicated prompt instructing it to score a given answer, for a specific question, along the three qualitative dimensions listed above. The model returned the scores in a structured dictionary format, allowing their storage and subsequent aggregation.

All metrics were computed for each instruction–prediction pair produced by the three models, and the resulting values were used to generate bar charts representing the average score obtained by the fine-tuned model, the RAG-KG model, and the zero-shot LLM baseline.

Each metric was calculated by comparing the predicted answer with its corresponding reference answer in the test split of the instruction-response dataset. Using the test set as the evaluation ground naturally favored the fine-tuned model, since it was trained on data from the same distribution (using the training split of the same dataset). To mitigate this

bias, the automatic metrics were computed both with and without stopwords, in order to more accurately capture the performance of the baseline models, whose outputs tended to be more verbose, a characteristic that can negatively affect certain string-matching metrics.

The results obtained from the automatic metrics indicate a clear performance advantage of the fine-tuned model over the two baseline models. Figure 9 shows that the F1 score for factoid questions is substantially higher for the fine-tuned model compared to both the RAG-KG model and the zero-shot LLM. Likewise, the ROUGE scores reported in Figures 10, 11 and 12, as well as the BLEU scores in Figures 13, 14, 15, and 16, consistently demonstrate superior performance for the fine-tuned model.

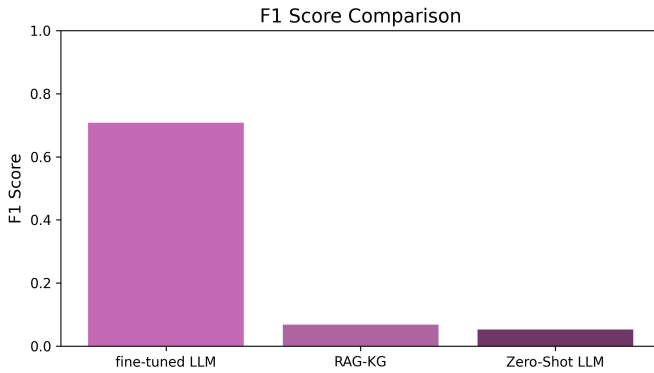


Fig. 9. F1 scores on three comparison models.

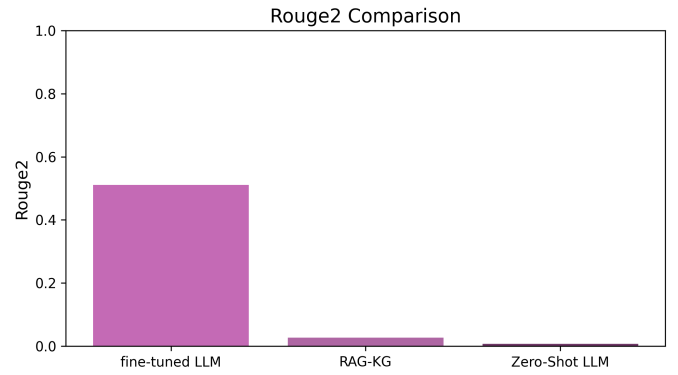


Fig. 11. ROUGE2 scores on three comparison models.

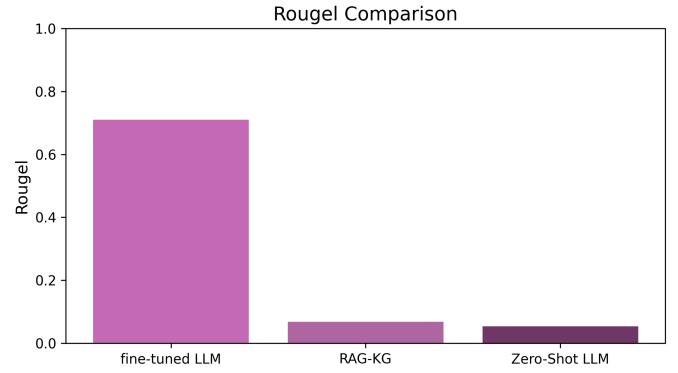


Fig. 12. ROUGE L scores on three comparison models.

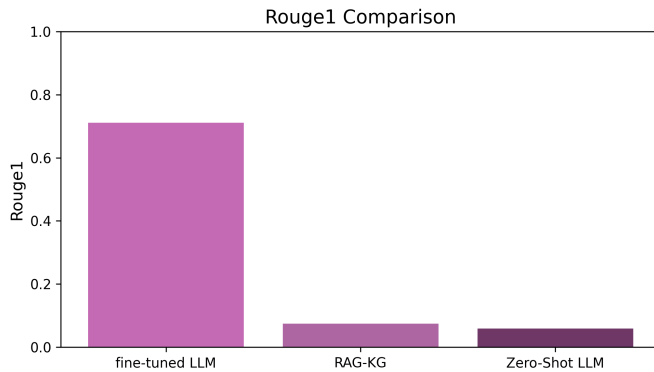


Fig. 10. ROUGE1 scores on three comparison models.

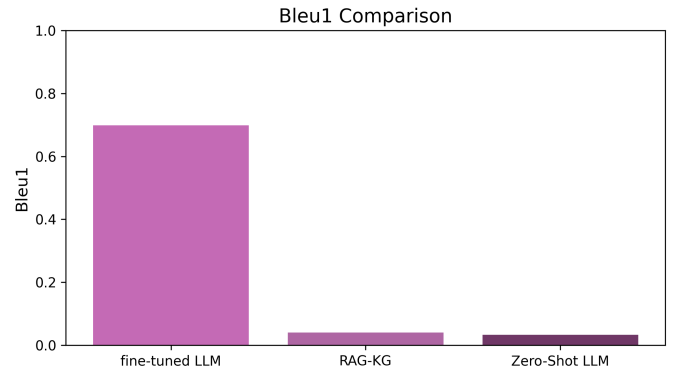


Fig. 13. BLEU1 scores on three comparison models.

Figure 17 shows the constraint satisfaction rate of the three models. A deeper manual evaluation reveals that the lower performance of the fine-tuned LLM stems from its tendency to include as much relevant information as possible, such as listing three food nutrients when only two are required.

These results are strongly skewed in favor of the fine-tuned model partly because it is the only one trained on data written in the same format as the answers in the test set, and is therefore able to reproduce the expected output structure with higher coherence. In contrast, automatic metrics are not capable of reliably assessing the semantic correctness

or completeness of an answer: a response may be accurate from a content perspective, but if written in a different format (e.g., more verbose or narrative), it may receive very low scores under metrics such as F1 score.

Moreover, several instructions in the test set admitted more than one valid answer. In such cases, any response that was correct but different from the reference answer negatively affected the automatic metric results, further penalizing models that generated plausible yet non-identical outputs.

After a combined manual evaluation and an external LLM-based assessment, it was confirmed that the fine-tuned model

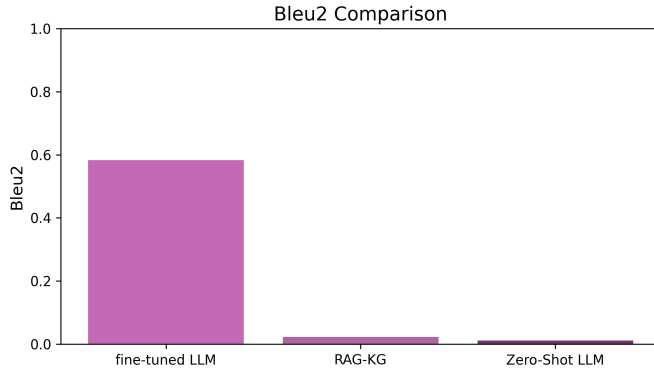


Fig. 14. BLEU2 scores on three comparison models.

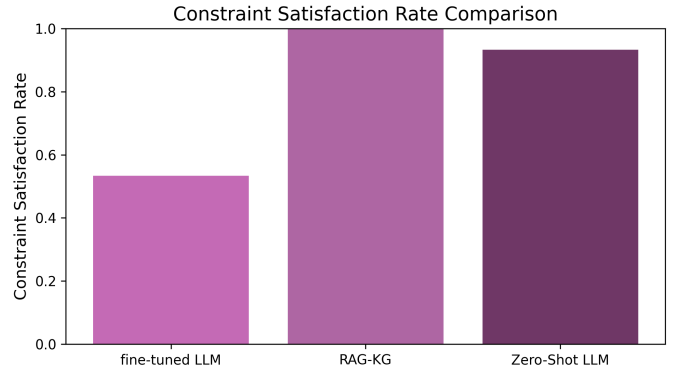


Fig. 17. Constraint satisfaction rate on three comparison models.

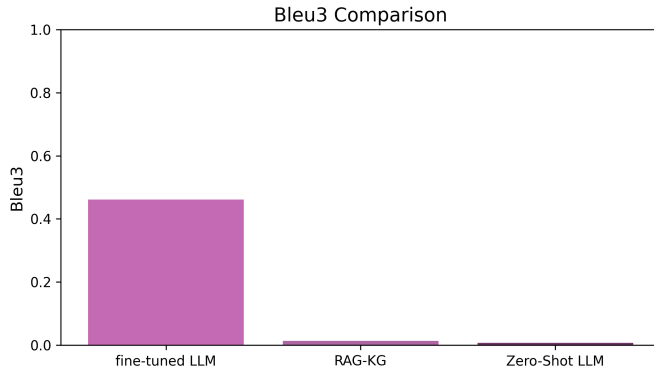


Fig. 15. BLEU3 scores on three comparison models.

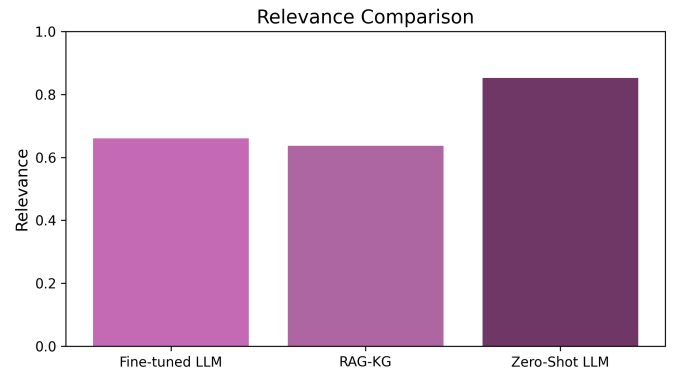


Fig. 18. Relevance on three comparison models.

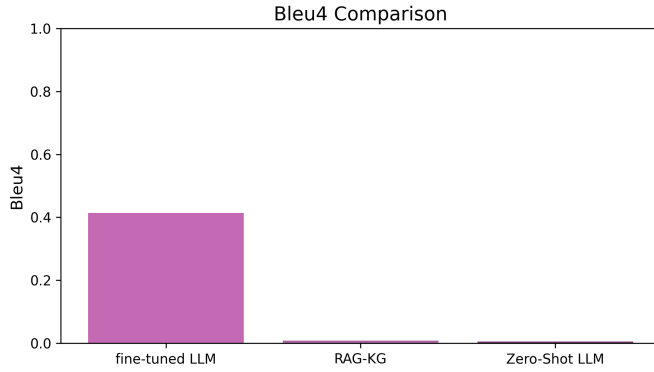


Fig. 16. BLEU4 scores on three comparison models.

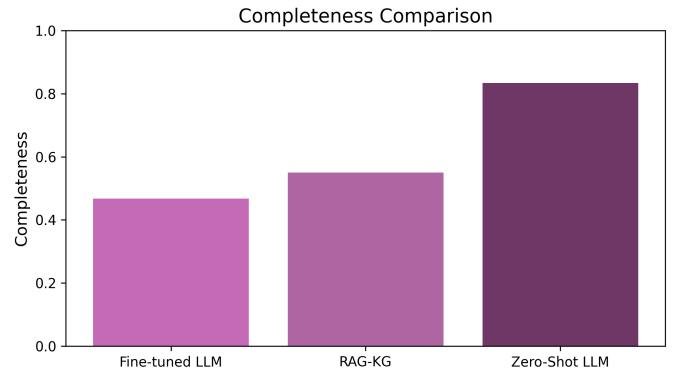


Fig. 19. Completeness on three comparison models.

achieves performance comparable to, or better than, both the RAG-KG model and the zero-shot LLM, despite having been trained on only a single data source (a PDF document). As shown in Figures 18, 19 and 20 the rubric-based metrics confirmed the reliability and overall quality of the fine-tuned model's outputs.

#### K. Hallucination detection and mitigation

In the hallucination detection and mitigation phase, the hallucination rate of each of the three models was evaluated. The hallucination rate refers to the frequency with which a

model generates false, fabricated, or unsupported information while maintaining a tone of confidence. A hallucination occurs when a model produces output that is factually incorrect, not grounded in input data or inconsistent with real-world knowledge.

$$\text{Hallucination Rate} = \frac{\text{Number of hallucinated responses}}{\text{Total responses evaluated}}$$

For this phase, hallucination detection focused exclusively on output content that could not be verified in the input

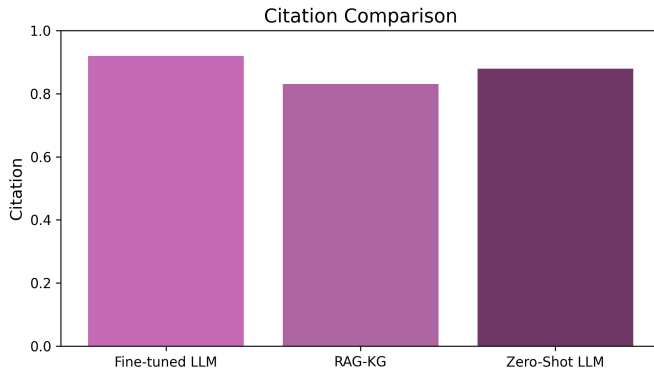


Fig. 20. Citation correctness on three comparison models.

data. For this purpose, subject–predicate–object triples were extracted from each generated response and checked against the knowledge graph. A response was classified as hallucinated if **at least one triple was not present in the knowledge graph**, indicating that the model introduced information not supported by the available evidence. The resulting value was converted into a percentage, producing the outcomes shown in Figure 21. As illustrated, the fine-tuned LLM once again achieved the lowest hallucination rate, demonstrating the most reliable behavior among the three models.

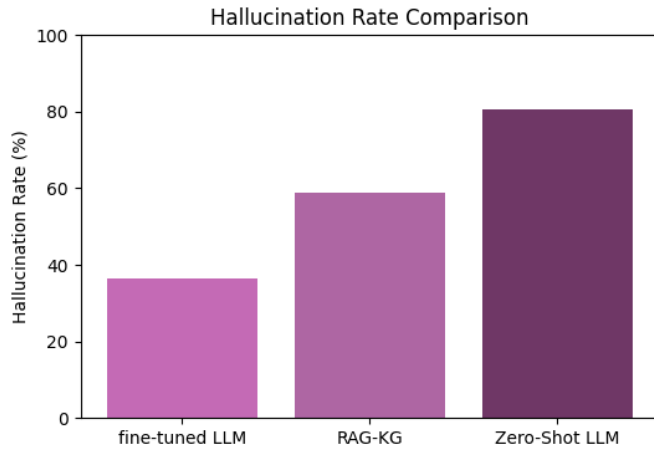


Fig. 21. Hallucination rate % on three comparison models.

Finally, after inspecting the identified hallucinations, it was observed that for the zero-shot LLM, several instances were classified as hallucinations, although they represented factual relations that simply were not present in the PDF source and, consequently, not included in the knowledge graph. Therefore, the actual hallucination rates of Zero-shot baseline models would likely be lower than reported, although still higher than that of the fine-tuned LLM.

### L. Delivery and demo

To illustrate the practical utility of the developed system, an interactive question–answering demo was implemented using

the Gradio framework. The interface allows users to query the knowledge base in three operational modes: Zero-shot, RAG-KG, and Fine-tuned. User input is processed through back-end functions that call the corresponding reasoning pipeline, returning both an answer and its grounding sources (page references from the original document). This demo enables direct comparison between model modes and demonstrates how structured knowledge from our reference document can be transformed into interpretable grounded responses.

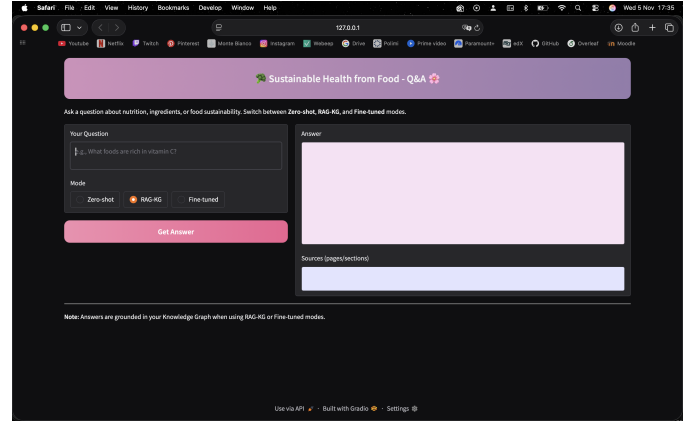


Fig. 22. UI demo, initial window.

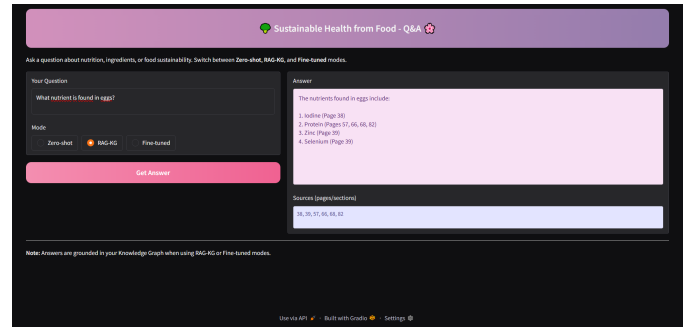


Fig. 23. Example of use of RAG-KG to answer question.

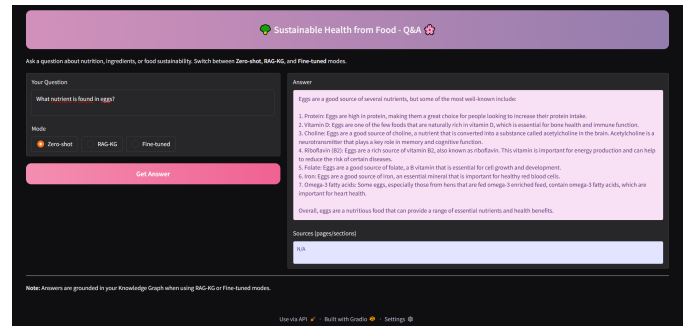


Fig. 24. Example of use of Zero-Shot to answer question.

## III. RESULTS AND DISCUSSIONS

The outcomes of this project demonstrate the feasibility and effectiveness of transforming a complex, domain-specific

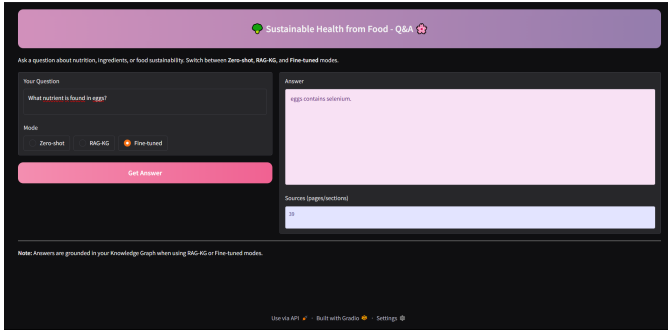


Fig. 25. Example of use of our fine tuned model to answer question.

report into a structured, machine-interpretable knowledge base. Starting from the Sustainable Health from Food – National Nutrition Recommendations 2024 report [1], the developed pipeline successfully extracted, organized, and operationalized knowledge for downstream use in language model grounding and evaluation.

The automated PDF ingestion and parsing process produced a well-structured dataset combining narrative and tabular elements. By leveraging the table of contents to preserve document hierarchy, the system maintained contextual integrity across sections while avoiding the fragmentation typically caused by page based extraction. The resulting dataset provided a faithful digital representation of the original report, forming a strong foundation for subsequent analytical stages.

The design of the domain ontology proved essential in organizing the extracted information into meaningful categories. Defining entities such as ingredient, nutrient, dietary guideline, and health outcome allowed the project to bridge textual information with formal semantic structure. This ontology not only guided the entity and relation extraction phases but also ensured conceptual consistency.

Entity and relation extraction yielded a good representation of the report’s contents. Thousands of entity mentions were mapped into typed relations. Although some ambiguity remained around overlapping or generic terms, the overall comprehensiveness of extracted relations was good, and this showed how also policy-oriented reports can be formalized into RDF-style triples. The resulting knowledge graph allowed queries that connect, for instance, food items with both their nutritional properties and associated health outcomes, thus offering systematic way to explore dietary recommendations.

Based on the structured knowledge graph, the generation of grounded instruction–response pairs provided a natural language layer that connected formal data with conversational tasks. This dataset demonstrated how triples could be rephrased into usable prompts and answers for LLM and evaluation. The inclusion of grounding metadata, such as source page numbers, strengthened transparency and interpretability.

Starting from the instruction–response dataset, the performance of the large language model (LLM) fine-tuned on the dataset’s contextual information was analyzed and compared with the two baseline models: a context-free LLM and a

RAG model enriched through the developed knowledge graph. The evaluation revealed the strong performance of the fine-tuned model, which, despite being trained on a single source, demonstrated the ability to effectively learn and reproduce the topic contained in the source PDF. As detailed in the evaluation section, the outcomes of the automatic metrics were primarily influenced by the output format generated by the three models, favoring the fine-tuned LLM, whose responses exhibited a stylistic coherence with those of the instruction–response dataset. The most noteworthy findings emerged from the rubric-based metrics, which enabled a more precise assessment of the substantive quality of the responses. These results indicated that the fine-tuned model achieved the highest score in Citation Correctness (Fig. 20) and performed well in Relevance (Fig. 18). Conversely, the presence of highly concise responses in the instruction–response dataset led the fine-tuned model to adopt a similarly succinct style, which ultimately penalized its performance in the Completeness metric (Fig. 19).

Finally, the interactive demo illustrated the end-to-end usability of the system. By allowing users to ask natural language questions and view the model’s grounded answers alongside their document citations, the interface made the abstract process of knowledge extraction and retrieval tangible. This proved valuable not only as a demonstration tool but also as a validation mechanism, showing that domain-grounded AI can serve as a bridge between expert reports and public-facing, interpretable systems.

Overall, the results highlight the potential of integrating structured knowledge extraction, semantic modelling, and grounded language generation in complex expert domains. The project illustrates how authoritative but static documents can be converted into dynamic information systems, ultimately contributing to more transparent and accountable applications of AI in health and nutrition.

#### IV. OVERALL DISCUSSIONS

The integrated pipeline demonstrates how structured knowledge extraction and large language model reasoning can be combined in a reproducible and interpretable framework. By connecting ontology-driven semantic structures to downstream generative tasks, the system bridges symbolic knowledge representation and data-driven natural language understanding. This approach enables not only accurate information retrieval but also transparent and traceable reasoning, as every model output can be linked back to its source document.

#### V. CONCLUSION

In recent years, the combination of structured knowledge representation and large language models has opened new opportunities for building systems that are both intelligent and interpretable. Ontologies and knowledge graphs play a crucial role in this space, providing the foundation for organizing complex domain knowledge in a transparent and reusable way. When integrated with generative models, they enable a new form of reasoning, one that combines the precision

of structured data with the flexibility of natural language understanding. In domains such as food, nutrition, and sustainability, this integration is particularly important. These areas require not only factual accuracy, but also contextual understanding, scientific traceability, and sensitivity to public health and environmental concerns. Structuring information about ingredients, nutrients, and health outcomes in a formal schema allows knowledge to be shared, queried, and validated across systems, supporting more consistent and evidence-based communication. More broadly, this direction illustrates a shift toward AI systems that are not just powerful, but explainable and accountable. By grounding model outputs in well-defined knowledge structures, we move closer to applications that support education, policy, and decision-making in transparent and trustworthy ways.

#### REFERENCES

- [1] N. N. Council, F. I. for Health, and Welfare, *Sustainable Health from Food – National Nutrition Recommendations 2024*. Helsinki: Finnish Institute for Health and Welfare, 2024, online publication. [Online]. Available: [https://www.ruokavirasto.fi/globalassets/teemat/terveyttiedistava-ruokavalio/ravitsemus-ja-ruokasuositukset/sustainable-health-from-food\\_web.pdf](https://www.ruokavirasto.fi/globalassets/teemat/terveyttiedistava-ruokavalio/ravitsemus-ja-ruokasuositukset/sustainable-health-from-food_web.pdf)
- [2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, *LoRA: Low-Rank Adaptation of Large Language Models*. Online: arXiv, 2021, arXiv preprint arXiv:2106.09685. [Online]. Available: <https://arxiv.org/abs/2106.09685>